

UNIVERSIDADE FEDERAL DE SERGIPE
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

RECUPERAÇÃO EFICIENTE DE CONTEXTO EM ARTIGOS CIENTÍFICOS SOBRE MODELOS DE LINGUAGEM, RECUPERAÇÃO DE INFORMAÇÃO E RETRIEVAL-AUGMENTED GENERATION (RAG): SELEÇÃO DE CONTEXTO COM PROGRAMAÇÃO DINÂMICA

CLARIANE MARIA OLIVEIRA LISBOA
IRIS AZENATE FERREIRA
JOSÉ NILTON DOS SANTOS OLIVEIRA
LEONARDO SOUZA SILVA
THIAGO DE JESUS DOS SANTOS
VICTÓRIA CARDOSO DOS SANTOS

São Cristóvão/SE
2026

INTRODUÇÃO



Íris Azenate Ferreira

- Na AV1: buscamos e ranqueamos artigos sobre RAG na ACL Anthology.
- Problema: o LLM só lê uma quantidade limitada de tokens, e a busca traz mais do que cabe.
- Na AV2: escolher quais artigos enviar, comparando ranking da AV1, guloso e programação dinâmica.

Corpus e Preparação



Íris Azenate Ferreira

```
import json
import math
import platform
import random
import re
import subprocess
import sys
import time
import tracemalloc
from collections import Counter, defaultdict
from pathlib import Path
import importlib.metadata as metadata
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import tiktoken
from IPython.display import display
from nltk.stem import PorterStemmer

SEMENTE = 42
SEED = SEMENTE   # nome usado nos testes
random.seed(SEMENTE)
np.random.seed(SEMENTE)

PASTA_DADOS = Path('dados')
PASTA_SAIDA = Path('resultados_av2')
PASTA_DADOS.mkdir(exist_ok=True)
PASTA_SAIDA.mkdir(exist_ok=True)

ARQUIVO_CORPUS = PASTA_DADOS / 'acl_corpus_av1.csv'
N_ARTIGOS_COLETA = 18000
ESCALAS_CORPUS = [500, 1000]    # duas quantidades de artigos
ESCALAS_CANDIDATOS = [20, 50]   # quantos resultados chegam à mochila
REPETICOES = 3
CONSULTAS_EXPERIMENTO = ['Q1', 'Q3', 'Q5']

TOKENIZADOR_NOME = 'cl100k_base'
ENC = tiktoken.get_encoding(TOKENIZADOR_NOME)
```

```
QUERY_TERMS = [
    'retrieval augmented generation',
    'information retrieval',
    'large language model'
]

QUERIES = {
    'Q1': 'What information retrieval methods are used in RAG systems?',
    'Q2': 'Which studies compare sparse retrieval and dense retrieval?',
    'Q3': 'What semantic search techniques are used to retrieve context for language models?',
    'Q4': 'What reranking methods are used in retrieval systems for LLMs?',
    'Q5': 'What metrics are used to evaluate Retrieval-Augmented Generation systems?',
    'Q6': 'Which papers use embeddings for document retrieval?',
    'Q7': 'What techniques are used to select the most relevant documents or excerpts for a language
model?',
    'Q8': 'Which methods were compared for context retrieval?',
    'Q9': 'What limitations of RAG systems are reported in the articles?',
    'Q10': 'What chunking strategies are used in document retrieval?'
}

print('Sistema:', platform.platform())
print('Python:', platform.python_version())

try:
    commit = subprocess.check_output(
        ['git', 'rev-parse', 'HEAD'],
        text=True,
        stderr=subprocess.DEVNULL
    ).strip()
except Exception:
    commit = 'preencher após publicar no GitHub'

print('Commit:', commit)

def contar_termos(texto, termos):
  """Pontuação para filtrar artigos na coleta inicial."""
  return sum(texto.count(termo) for termo in termos)


def baixar_artigos_acl(termos, quantidade):
  from acl_anthology import Anthology
```

```
  biblioteca = Anthology.from_repo()
  encontrados = []
  for artigo in biblioteca.papers():
    if artigo.title is None or artigo.abstract is None:
      continue

    titulo = str(artigo.title)
    resumo = str(artigo.abstract)
    pontos = contar_termos((titulo + ' ' + resumo).lower(), termos)
    if pontos > 0:
      encontrados.append({
        'id': str(artigo.full_id), 'title': titulo,
        'abstract': resumo, 'year': artigo.year,
        'doi': str(artigo.doi or ''), 'url': str(artigo.web_url),
        'pontos_coleta': pontos
      })

  encontrados.sort(key=lambda x: (x['pontos_coleta'], int(x['year'] or 0)), reverse=True)
  return pd.DataFrame(encontrados[:quantidade]).drop(columns='pontos_coleta')
if ARQUIVO_CORPUS.exists():
  df = pd.read_csv(ARQUIVO_CORPUS, dtype={'id': str})
  origem = 'CSV da AV1'
else:
  try:
    df = baixar_artigos_acl(QUERY_TERMS, N_ARTIGOS_COLETA)
  except Exception as erro:
    raise RuntimeError('Coloque o CSV real da AV1 em dados/acl_corpus_av1.csv') from erro
  df.to_csv(ARQUIVO_CORPUS, index=False)
  origem = 'Nova coleta da ACL Anthology'

colunas = {'id', 'title', 'abstract', 'year', 'url'}
assert colunas.issubset(df.columns), 'O CSV não contém todas as colunas necessárias.'
df = df.dropna(subset=['id', 'title', 'abstract'])
df = df.drop_duplicates(subset='id').reset_index(drop=True)
assert len(df) >= max(ESCALAS_CORPUS), 'São necessários pelo menos 1000 artigos reais.'

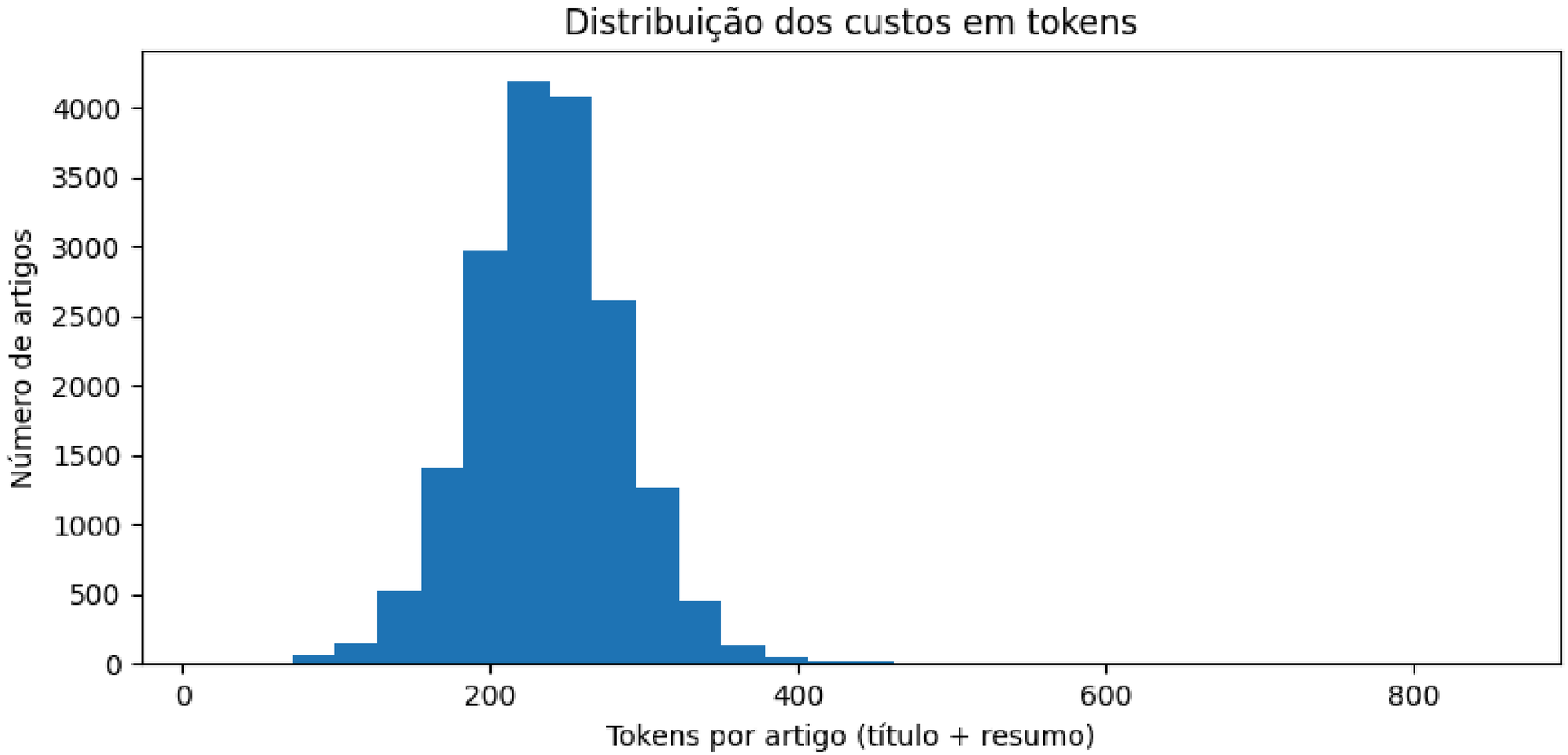
print('Origem:', origem, '| Artigos:', len(df))
display(df[['id', 'title', 'year']].head())
```

Custo em tokens



Íris Azenate Ferreira

-



PROBLEMA DA MOCHILA



Clariane Maria Oliveira Lisboa

- *"Tenho um limite máximo de tokens W (janela de contexto) e vários artigos candidatos. Quais artigos devo escolher para maximizar a relevância total sem estourar o limite de tokens?"*

Problema da mochila 0/1

- Capacidade da mochila: Limite de tokens do prompt (*W*).
- Peso do item: Custo em tokens de cada artigo (*cost*).
- Valor do item: Pontuação lexical de relevância (*value*).

Antes de começar precisaremos de alguns pré-requistos ou validações.

PRÉ-REQUISITO 1: VALIDAR ENTRADA



Clariane Maria Oliveira Lisboa

```python
def validar_entrada(itens, W):
    """Evita entradas que quebrariam a mochila."""

    if not isinstance(W, (int, np.integer)) or W < 0:
        raise ValueError("W deve ser inteiro e não negativo")

    ids = [str(a["id"]) for a in itens]
    if len(ids) != len(set(ids)):
        raise ValueError("IDs duplicados")

    for artigo in itens:
        if not isinstance(artigo["cost"], (int, np.integer)) or artigo["cost"] <= 0:
            raise ValueError("Tokens devem ser inteiros positivos")

        if not np.isfinite(artigo["value"]) or artigo["value"] < 0:
            raise ValueError("Pontuação inválida")
```

PRÉ-REQUISITO 2: DICIONÁRIO



Clariane Maria Oliveira Lisboa

```
def resposta(escolhidos):
    return {
        'ids': [a['id'] for a in escolhidos],
        'tokens': sum(int(a['cost']) for a in escolhidos),
        'valor': sum(float(a['value']) for a in escolhidos),
        'n': len(escolhidos)
    }
```

Exemplo do dicionário final:

'ids': ['ACL_01', 'ACL_02'],

'tokens': 250,

'valor': 13.0,

'n': 2

MÉTODOS DE SELEÇÃO



José Nilton dos Santos Oliveira

O problema da mochila é um problema ***NP-difícil***, isso significa que não é conhecida uma solução em tempo polinomial, por isso usaremos métodos para resolver o problema, para efeitos comparativos trouxemos 4 métodos de solução desse problema ***NP-difícil***.

- Abordagem por Ranking Simples (Baseline);
- Abordagem Gulosa;
- Programação Dinâmica (Mochila 0/1);
- Abordagem Exaustiva.

ABORDAGEM POR RANKING SIMPLES



José Nilton dos Santos Oliveira

```python
def ranking_av1(itens, W):
    """Baseline: tenta os artigos com maiores pontuações primeiro."""
    validar_entrada(itens, W)
    ordem = sorted(itens, key=lambda a: (-a['value'], a['id']))
    escolhidos = []
    espaco_livre = W
    for artigo in ordem:
        if artigo['value'] > 0 and artigo['cost'] <= espaco_livre:
            escolhidos.append(artigo)
            espaco_livre -= artigo['cost']
    return resposta(escolhidos)
```

MÉTODO GULOSO



José Nilton dos Santos Oliveira

```python
def guloso_razao(itens, W):
    """Guloso: prefere maior pontuação dividida pelo custo."""
    validar_entrada(itens, W)
    ordem = sorted(itens, key=lambda a: (-a['value']/a['cost'], -a['value'], a['id']))
    escolhidos = []
    espaco_livre = W
    for artigo in ordem:
        if artigo['value'] > 0 and artigo['cost'] <= espaco_livre:
            escolhidos.append(artigo)
            espaco_livre -= artigo['cost']
    return resposta(escolhidos)
```

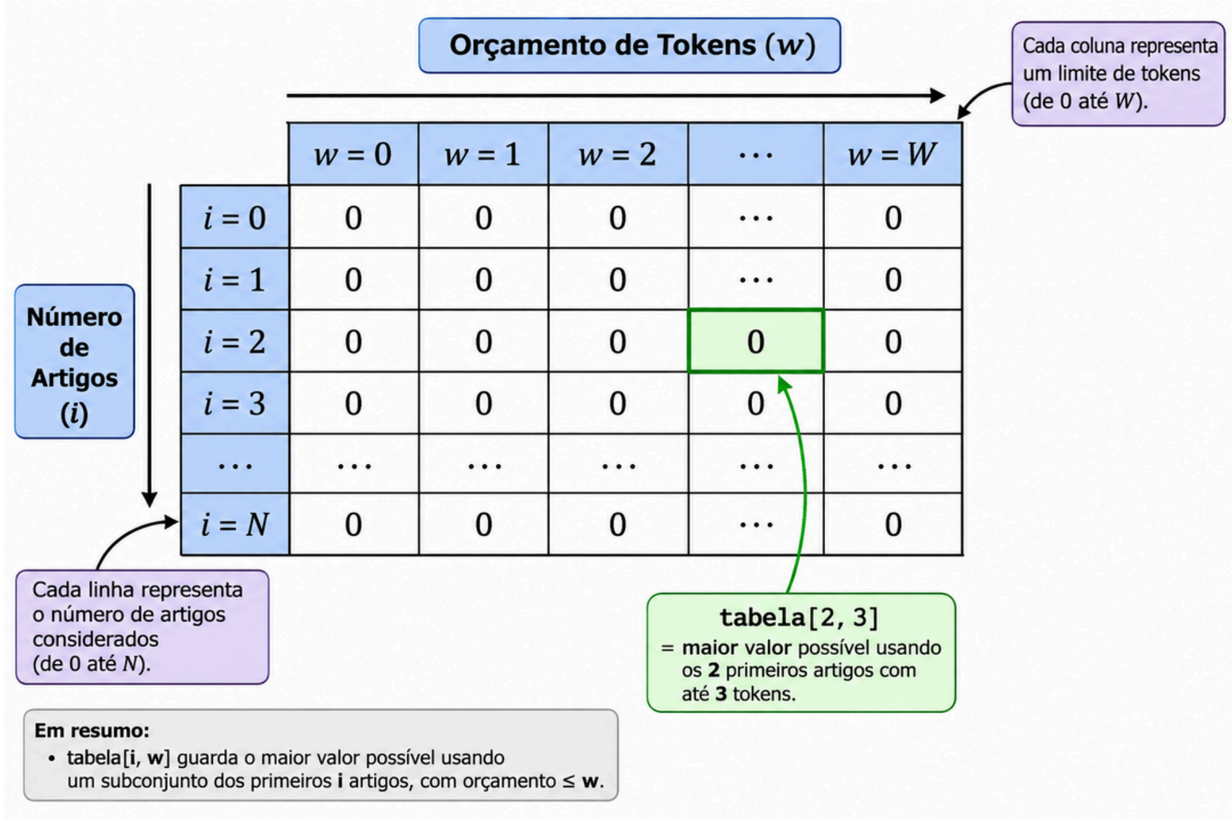
PROGRAMAÇÃO DINÂMICA



Leonardo Souza Silva

```python
def mochila_dp(itens, W):
    """Programação dinâmica: encontra a melhor soma de pontuações."""
    validar_entrada(itens, W)
    quantidade = len(itens)

    # Tabela de valores e tabela para lembrar quais artigos foram escolhidos.
    tabela = np.zeros((quantidade + 1, W + 1))
    escolhido = np.zeros((quantidade + 1, W + 1), dtype=bool)
```



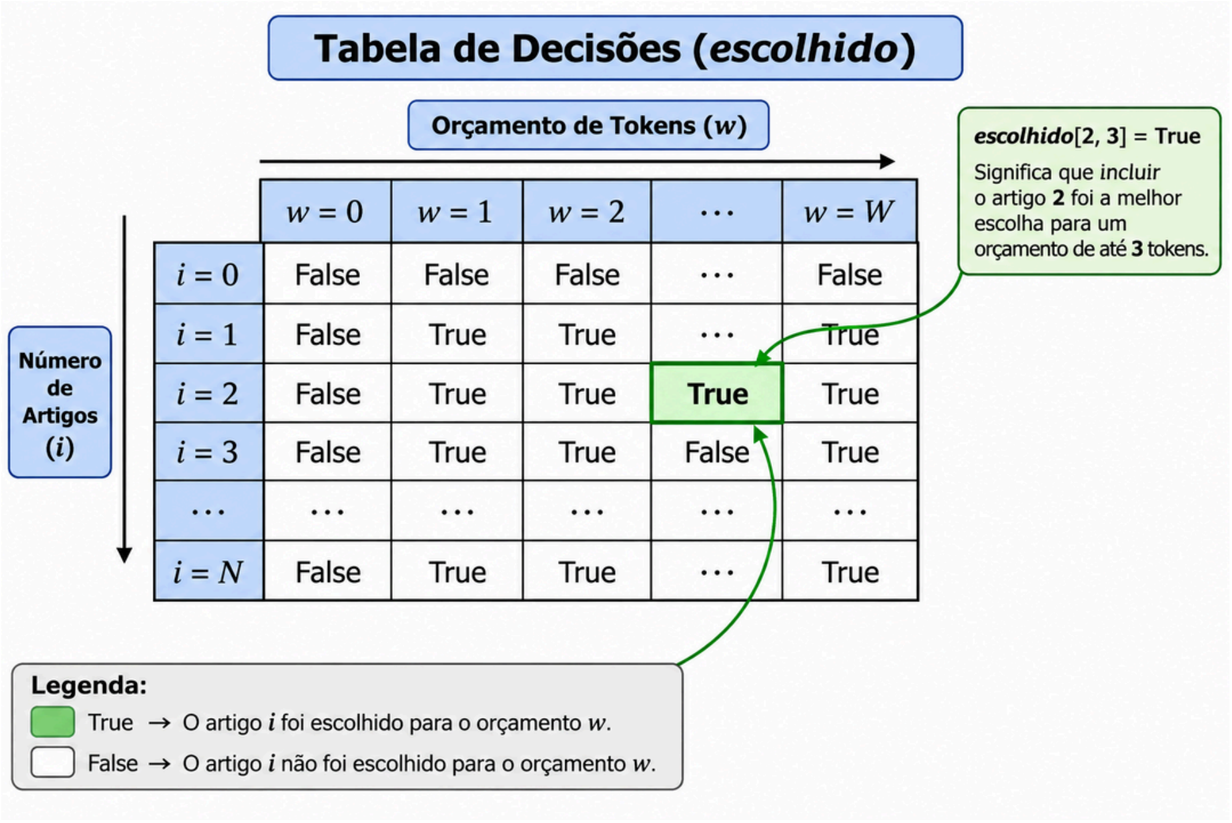
PROGRAMAÇÃO DINÂMICA



Leonardo Souza Silva

```python
def mochila_dp(itens, W):
    """Programação dinâmica: encontra a melhor soma de pontuações."""
    validar_entrada(itens, W)
    quantidade = len(itens)

    # Tabela de valores e tabela para lembrar quais artigos foram escolhidos.
    tabela = np.zeros((quantidade + 1, W + 1))
    escolhido = np.zeros((quantidade + 1, W + 1), dtype=bool)
```



PROGRAMAÇÃO DINÂMICA



Leonardo Souza Silva

```
for i, artigo in enumerate(itens, start=1):
    custo = int(artigo['cost'])
    pontos = float(artigo['value'])

    # Começa com a opção de NÃO pegar o artigo atual.
    tabela[i] = tabela[i - 1]
    if custo > W or pontos == 0:
        continue

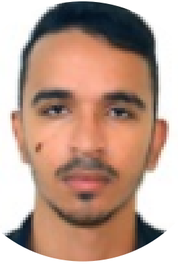
    # Se pegar o artigo, o valor é o anterior + pontos dele.
    # O fatiamento faz a comparação para todos os orçamentos de uma vez.
    valor_se_pegar = tabela[i - 1, :W + 1 - custo] + pontos
    valor_sem_pegar = tabela[i, custo:]
    vale_a_pena = valor_se_pegar > valor_sem_pegar + 1e-10

    tabela[i, custo:] = np.where(vale_a_pena, valor_se_pegar, valor_sem_pegar)
    escolhido[i, custo:] = vale_a_pena
```

```
# Caminha do final da tabela até o início para descobrir os IDs.
espaco = W
artigos_escolhidos = []
for i in range(quantidade, 0, -1):
    if escolhido[i, espaco]:
        artigo = itens[i - 1]
        artigos_escolhidos.append(artigo)
        espaco -= int(artigo['cost'])

artigos_escolhidos.reverse()
resultado = resposta(artigos_escolhidos)
assert resultado['tokens'] <= W
assert math.isclose(resultado['valor'], tabela[quantidade, W], abs_tol=1e-7)
return resultado
```

DEMONSTRAÇÃO SIMPLES



Leonardo Souza Silva

- Suponha uma mochila de peso 5, ou seja, ***w = 5***. E os seguintes artigos
  - *Artigo A (i = 1): Custo = 2, Valor = 3*
  - *Artigo B (i = 2): Custo = 3, Valor = 4*
  - *Artigo C (i = 3): Custo = 1, Valor = 2*

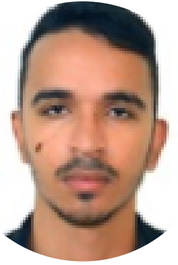
Como temos 3 artigos e peso 5, a matriz será (3+1) x (5+1)

O caso mais simples é i = 0, chamado caso base

Matriz tabela (*Valores Máximos*)

	w = 0	*w* = 1	*w* = 2	*w* = 3	*w* = 4	*w* = 5
i = 0 (Nenhum)	0	0	0	0	0	0

DEMONSTRAÇÃO SIMPLES



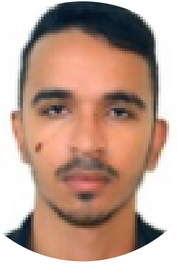
Leonardo Souza Silva

Entrada do Exemplo	
Capacidade da Mochila (*W*): **5 tokens**	**Artigos (itens):** **A** (*i* = 1): Custo = 2, Valor = 3 **B** (*i* = 2): Custo = 3, Valor = 4 **C** (*i* = 3): Custo = 1, Valor = 2

Matriz tabela (*Valores Máximos*)

	$w = 0$	$w = 1$	$w = 2$	$w = 3$	$w = 4$	$w = 5$
$i = 0$ (Nenhum)	0	0	0	0	0	0
$i = 1$ (Artigo A)	0	0	3	3	3	3
$i = 2$ (Artigo B)	0	0	3	4	4	7
$i = 3$ (Artigo C)	0	2	3	5	6	7

DEMONSTRAÇÃO SIMPLES



Leonardo Souza Silva

Entrada do Exemplo	
Capacidade da Mochila (*W*): **5 tokens**	**Artigos (itens):** **A** (*i* = 1): Custo = 2, Valor = 3 **B** (*i* = 2): Custo = 3, Valor = 4 **C** (*i* = 3): Custo = 1, Valor = 2

Matriz tabela (*Valores Máximos*)

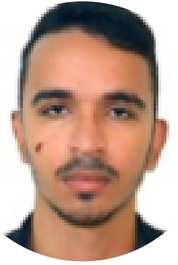
	$w = 0$	$w = 1$	$w = 2$	$w = 3$	$w = 4$	$w = 5$
$i = 0$ (Nenhum)	0	0	0	0	0	0
$i = 1$ (Artigo A)	0	0	3	3	3	3
$i = 2$ (Artigo B)	0	0	3	4	4	7
$i = 3$ (Artigo C)	0	2	3	5	6	7

Matriz escolhido (*Histórico de Decisão*)

	$w = 0$	$w = 1$	$w = 2$	$w = 3$	$w = 4$	$w = 5$
$i = 1$ (Artigo A)	False	False	True	True	True	True
$i = 2$ (Artigo B)	False	False	False	True	True	True
$i = 3$ (Artigo C)	False	True	False	True	True	False

✅ **Resultado Final: Valor Máximo = 7**
(Combinação: Artigos A + B, com custo total = 5)

ABORDAGEM EXAUSTIVA



Leonardo Souza Silva

```python
def exaustiva(itens, W):
    """Confere todas as combinações (somente para testes pequenos)."""
    validar_entrada(itens, W)
    if len(itens) > 20:
        raise ValueError('Use no máximo 20 artigos na busca exaustiva')

    melhor = resposta([])
    for mascara in range(2 ** len(itens)):
        escolhidos = [a for i, a in enumerate(itens) if mascara & (1 << i)]
        atual = resposta(escolhidos)
        if atual['tokens'] <= W and atual['valor'] > melhor['valor'] + 1e-10:
            melhor = atual
    return melhor
```

TESTES FUNCIONAIS E DE INTEGRIDADE DE SOFTWARE



Leonardo Souza Silva

- Teste padrão de uso máximo de tokens;
- Corpus Vazio;
- Tokens excedidos (orçamento zerado $W = 0$);
- Borda exata (Item cobre o orçamento exato de tokens);
- Item Inviável (Valor do item maior que o orçamento);
- Tratamento de empates (Dois itens exatamente do mesmo peso e custo);
- Artigo sem relevância ($Value = 0$);
- Alto orçamento de tokens (Deve escolher grande quantidade de artigos, sem afetar desempenho);
- Caso adversárial simples e pós-imediato (Para refutar o método guloso);
- Consulta sem correspondência;
- Rejeição de IDs duplicados;

OBS.: Todos esses testes estão disponíveis no nosso algoritmo do github:
https://github.com/Leonardodcomp/PAA_UFS_2026_2_Ferreira_Iris_Ferreira_Jose_Lisboa_Clariane_Santos_Thiago_Santos_Victoria_Silva_Leo

CORRETUDE



Leonardo Souza Silva

Conceitualmente, a corretude da Programação Dinâmica é garantida pela teoria (princípio da subestrutura ótima e indução sobre a recorrência de Bellman). Como o algoritmo exaustivo testa todas as 2N2^N2N combinações, ele serve como verdade absoluta para instâncias pequenas, confirmando que a implementação da DP alcançou 100% de concordância com o ótimo real.

```
rng = random.Random(SEED)
ok = True
for rodada in range(30):
    xs = [item(f'{rodada}-{j}', rng.randint(1,12), rng.randint(0,15)) for j in range(rng.randint(0,12))]
    w = rng.randint(0,40)
    if not math.isclose(mochila_dp(xs,w)['valor'], exaustiva(xs,w)['valor'], abs_tol=1e-7):
        ok = False; break
testes.append({
    'teste':'12. DP × exaustiva (30 instâncias)',
    'esperado':'mesmo valor ótimo nas 30',
    'observado':'30 concordâncias' if ok else f'divergência na rodada {rodada}',
    'status':'PASSOU' if ok else 'FALHOU',
    'requisito':'corretude'
})
```

DESENHO EXPERIMENTAL



Thiago de Jesus dos Santos

Corpus

500 e 1.000

artigos

Candidatos

20 e 50

por consulta

Orçamento W

708 e 1.416

tokens (3× e 6× a mediana)

Consultas

Q1, Q3, Q5

3 repetições cada

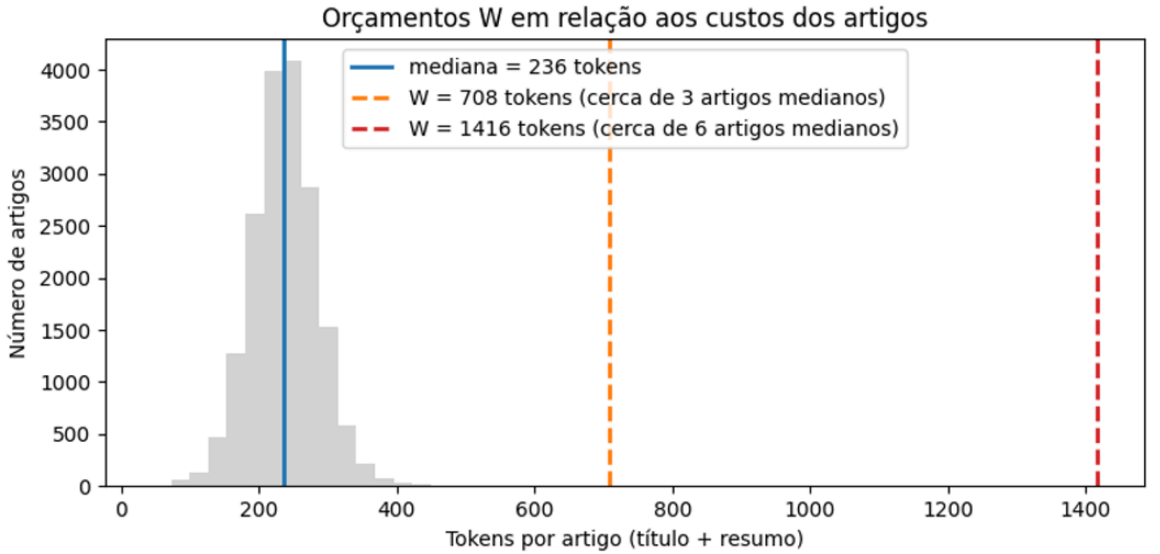


Figura 1 – Orçamentos em relação à distribuição dos custos (mediana = 236 tokens).

Métodos comparados

Ranking AV1 · Guloso por razão valor/custo · Programação dinâmica (mochila 0/1)

Métrica principal

Soma dos scores lexicais sob o limite de tokens

2 × 2 × 2 × 3 configurações × 3 repetições × 3 métodos = 216 medições

RESULTADOS



Thiago de Jesus dos Santos

Corpus	Método	Score	Tempo (ms)	Mem. (MB)	Tokens	Precisão	P@5	Jaccard
500	Ranking	60,33	0,235	0,0031	675,00	0,722	0,333	0,194
500	Guloso	72,67	0,246	0,0031	669,00	0,500	0,333	0,172
500	DP	74,67	1,318	0,3216	699,33	0,639	0,400	0,187
1.000	Ranking	60,33	0,231	0,0031	675,00	0,722	0,333	0,194
1.000	Guloso	72,00	0,240	0,0031	661,00	0,639	0,400	0,181
1.000	DP	74,67	1,196	0,3213	698,33	0,639	0,400	0,197

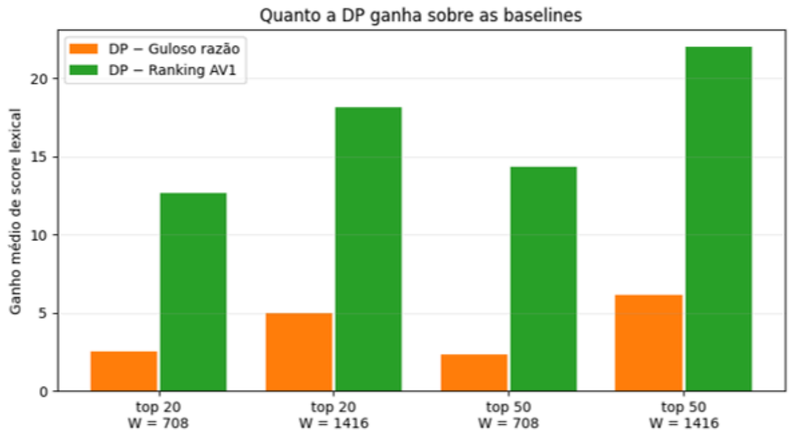


Figura 2 – Ganho médio da DP.

+21,0% sobre o Ranking AV1 e **+4,3%** sobre o Guloso (500 artigos, 20 candidatos, W = 708)

DP superou o Guloso em **48 de 72** combinações; nas demais, empate

ANÁLISE ASSINTÓTICA



Victória Cardoso dos Santos

Quadro 1 – Síntese da análise assintótica

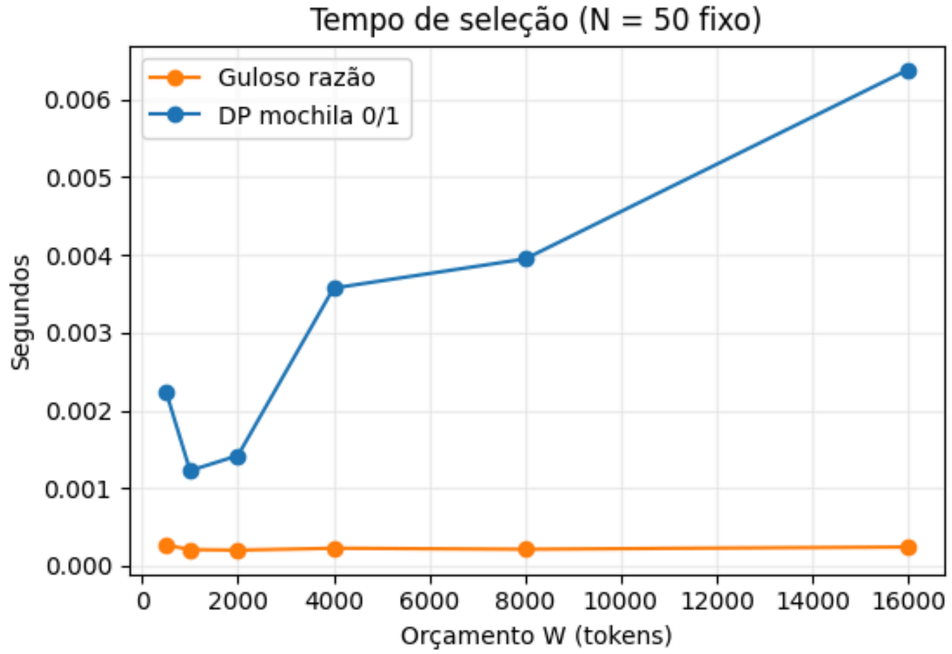
Algoritmo implementado	**Melhor caso**	**Pior Caso**	**Espaço**
Ranking AV1	$\Theta(N \log N)$	$\Theta(N \log N)$	$O(N)$
Guloso	$\Theta(N \log N)$	$\Theta(N \log N)$	$O(N)$
Programação Dinâmica	$\Theta(N \times W)$	$\Theta(N \times W)$	$\Theta(N \times W)$
Busca exaustiva	$\Theta(N \times 2^N)$	$\Theta(N \times 2^N)$	$O(N)$

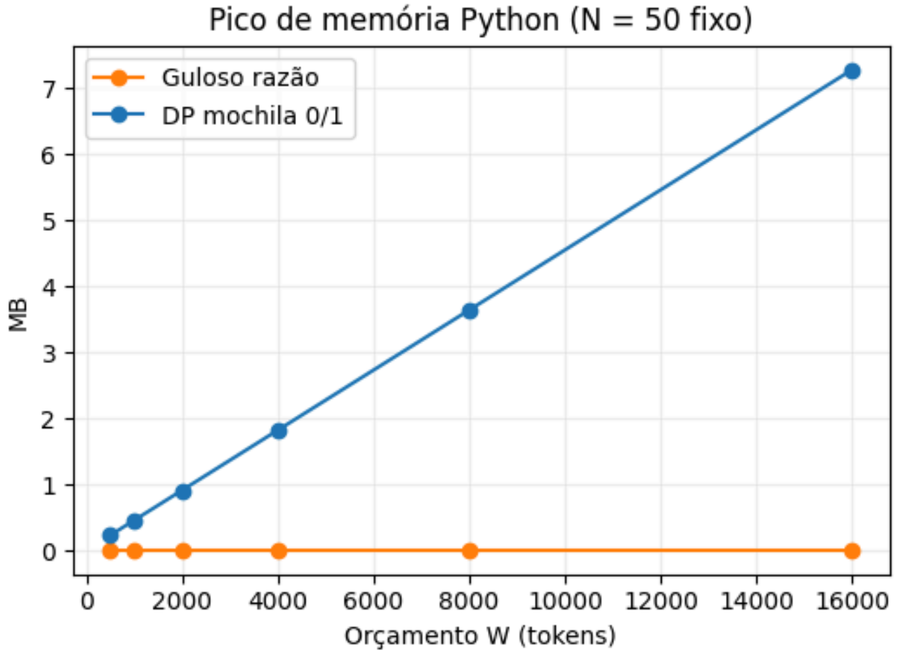
DP - Comportamento pseudopolinomial



Victória Cardoso dos Santos

- O tempo e a memória aumentam conforme cresce o orçamento de tokens (W).
- A complexidade O(N × W) é pseudopolinomial por depender do valor numérico de W.





Aplicação no RAG e Limitações



Victória Cardoso dos Santos

Aplicação no RAG

- Seleção de trechos relevantes dentro do orçamento de tokens.
- Organização do contexto enviado ao LLM.
- Melhor aproveitamento da janela de contexto.

Limitações

- Consumo de memória em orçamentos elevados.
- Dependência das pontuações de relevância.
- A maior pontuação não garante, por si só, a melhor resposta do LLM.

Conclusão



Victória Cardoso dos Santos

- A Programação Dinâmica encontra a combinação ótima para o modelo de seleção adotado.
- O Guloso apresenta menor custo computacional, mas não garante o ótimo.
- A escolha do algoritmo envolve um equilíbrio entre qualidade da seleção e custo computacional.